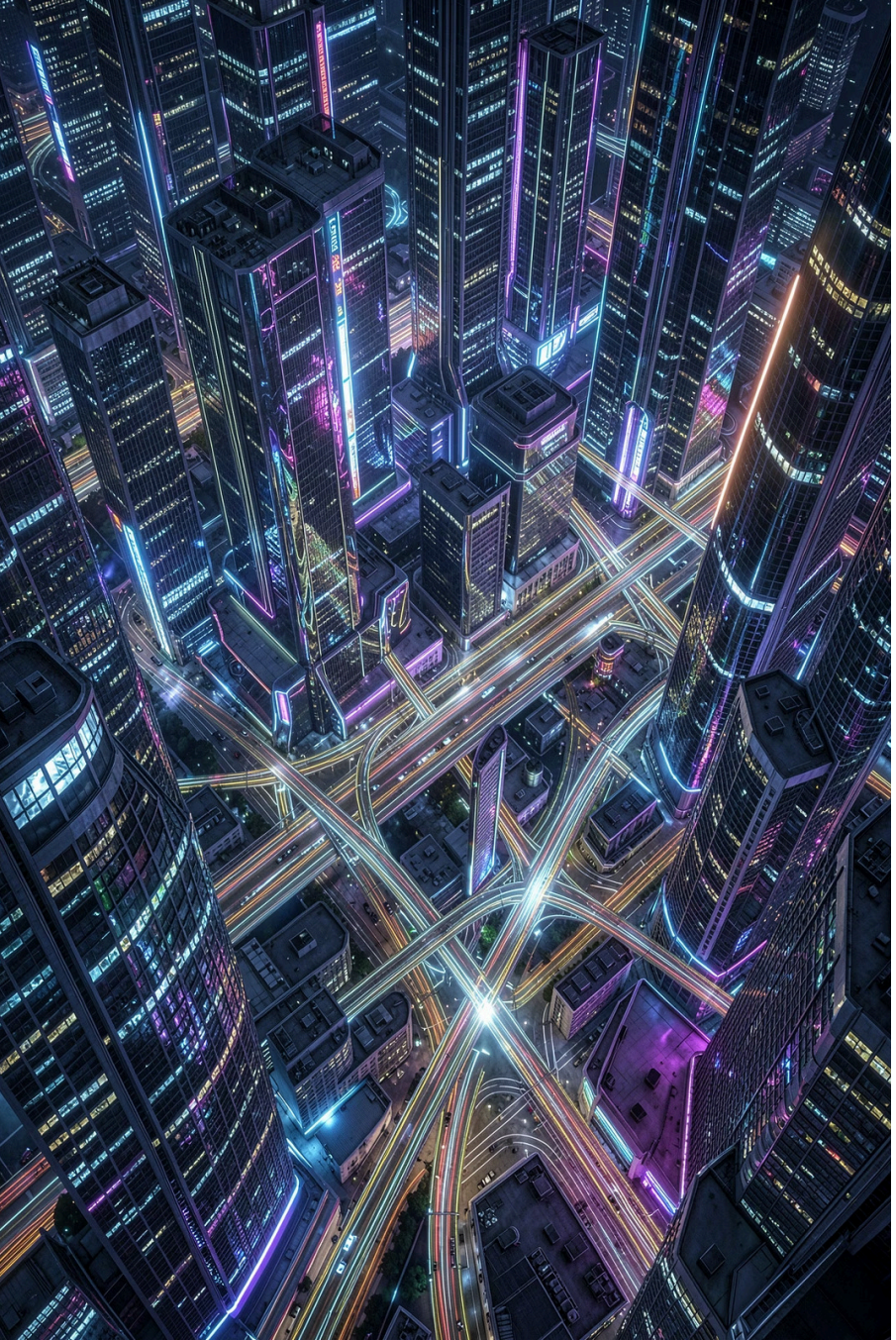


# Metropolis: City Routing Engine Hackathon

Graphs · Adjacency · BFS · DFS · Grid-as-Graph · Problem Statements

DEVELOPED BY TALENCIAGLOBAL

# About This Hackathon

Metropolis is a city routing engine built on graphs. Intersections are vertices and two-way roads are edges; a separate districts map is a grid where **1 is a city block** and **0 is open ground**. Across six levels you will map the network, find shortest hop counts, discover districts, treat the grid as a graph, route across town, and finally audit the whole network.

## Graph Input Format

Graphs are given as `N M` followed by M undirected edges `u v` (vertices are 0..N-1); some levels then read a source vertex. Adjacency lists are kept sorted so traversals are deterministic; distances are in edges (hops).

## Grid Input Format

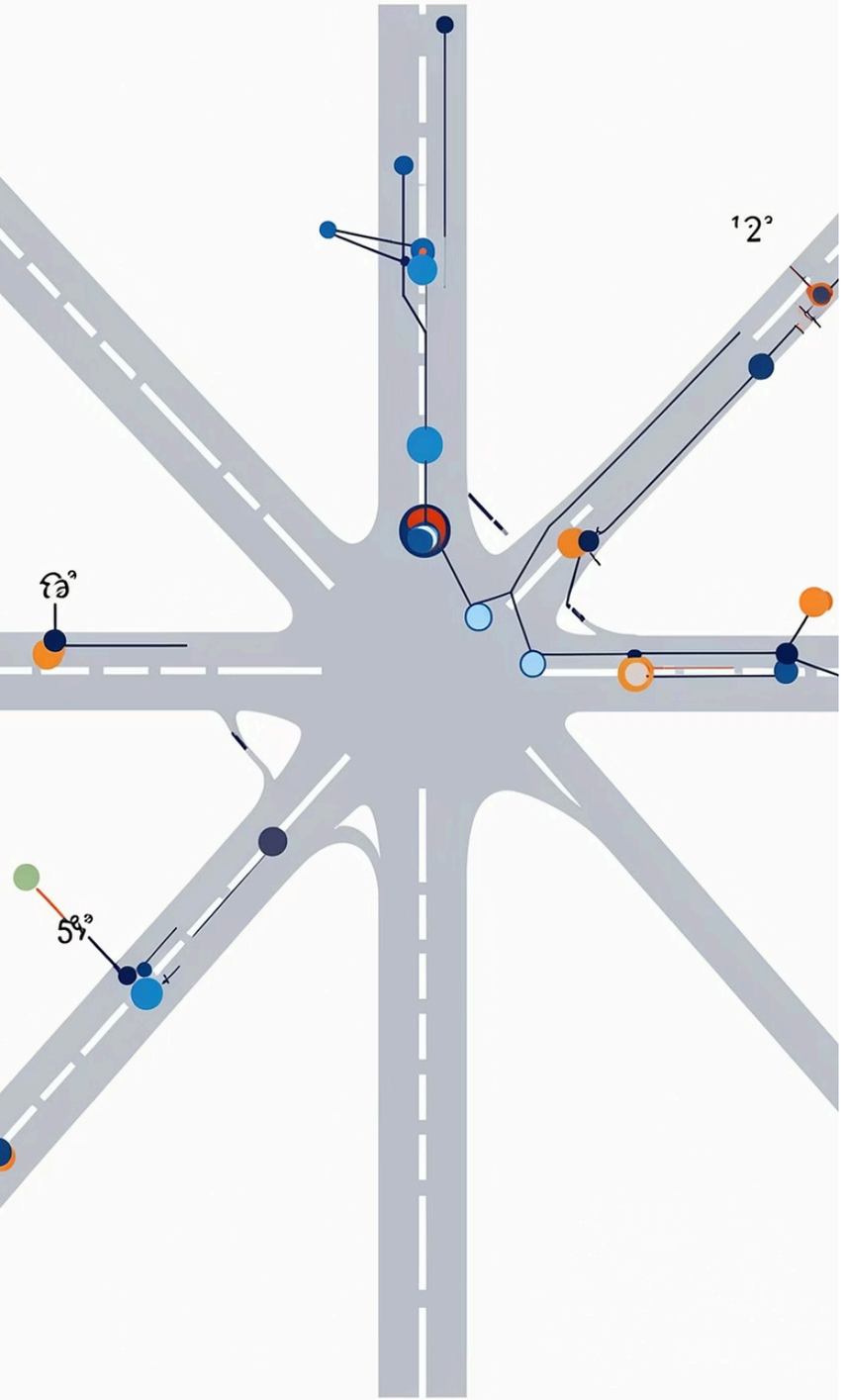
Grids are given as `R C` followed by R rows of C integers. The levels run from intermediate to expert. No starter code is provided — design and write each solution yourself.

# Six Levels at a Glance

The challenge escalates from foundational graph representation to a full network audit capstone.

| | | |
|---|---|---|
| 1 | **Map the City** | Adjacency & Degrees<br>★★★ INTERMEDIATE |
| 2 | **Shortest Hops** | Breadth-First Search<br>★★★ INTERMEDIATE |
| 3 | **Find the Districts** | DFS & Components<br>★★★★ ADVANCED |
| 4 | **City Blocks** | Matrix as Graph<br>★★★★ ADVANCED |
| 5 | **Cross Town** | Grid Shortest Path<br>★★★★ ADVANCED |
| 6 | **Network Audit** | Capstone Challenge<br>★★★★★ EXPERT |

tria'niols Bxwnedcuction lespeched.



LEVEL 1

# Map the City

## Graph Basics — Adjacency & Degrees · ★★★ Intermediate

Start by mapping the road network. Read the intersections and roads into an adjacency list, then report the network's basic shape: how many intersections and roads, the busiest intersection's degree, and how many intersections sit isolated with no road at all.

### Adjacency List

Store each undirected edge in both endpoints

### Vertex Degree

Count of neighbours for each vertex

### Max Degree

Busiest intersection in the network

### Isolated Vertices

Intersections with degree zero

# Level 1 — Problem Details

## Problem Statement

**Given:** A line `N M`, then M undirected edges `u v` (vertices are 0..N-1)

**Required output:**

- The number of vertices
- The number of edges
- The maximum degree over all vertices
- The number of isolated vertices (degree 0)

## Formula Guidance

`degree(v)` = number of neighbours of v
`max degree` = max over all v of degree(v)
`isolated` = count of v with degree 0

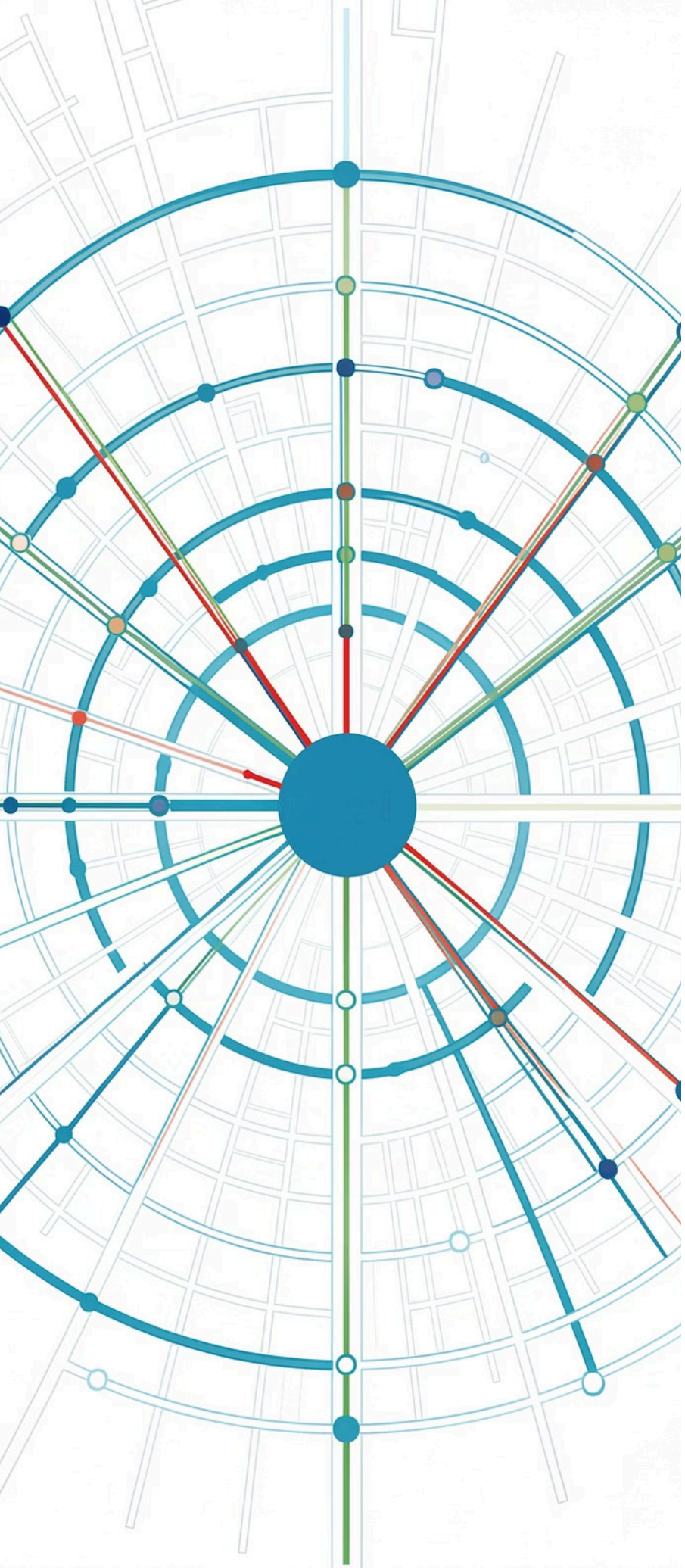
⚠ Add every undirected edge to **both** endpoints; storing it once understates degrees and miscounts isolated vertices.

## Sample Run

Input:
7 4
0 1
0 2
2 3
4 5

Output:
Vertices: 7
Edges: 4
Max Degree: 2
Isolated: 1

ⓘ An adjacency list is the workhorse graph representation; once built, structural facts like degree, max degree, and isolation are simple reads over the lists.



LEVEL 2

# Shortest Hops

## Breadth-First Search · ★★★ Intermediate

Find the shortest routes by hop count from a starting intersection. A breadth-first search expands the network ring by ring, so the first time it reaches an intersection is along a shortest path — report the BFS visiting order, how many intersections are reachable, and the farthest hop distance.

### BFS with a Queue

Expand neighbours ring by ring from the source vertex using a FIFO queue

### Shortest Paths

First visit to any vertex is guaranteed to be along a minimum-hop route

### Distance Array

Track hop distances from source; unreachable vertices remain at -1

### Reachability

Count all vertices with a finite distance from the source

# Level 2 — Problem Details

## Problem Statement

**Given:** A graph `N M` with M edges, then a source vertex S

**Required output:**

- The BFS visiting order from S
- The number of reachable vertices (including S)
- The farthest hop distance from S among reachable vertices

## Formula Guidance

`BFS:` dist[S]=0; pop u, for each unvisited neighbour w: dist[w]=dist[u]+1, enqueue w
`reachable` = count of vertices with dist != -1
`farthest` = max finite dist

⚠ Mark a vertex as seen when you **enqueue** it, not when you pop it — otherwise it can be queued multiple times and distances inflate.

## Sample Run

Input:
7 4
0 1
0 2
2 3
4 5
0

Output:
BFS Order: 0 1 2 3
Reachable: 4
Farthest: 2

ⓘ BFS gives shortest paths in unweighted graphs for free: because it expands in distance order, the first visit to a vertex is always along a minimum-hop route.

LEVEL 3

# Find the Districts

## Depth-First Search & Components · ★★★★ Advanced

Group the city into districts — maximal sets of intersections reachable from one another. A depth-first search floods each district; count the districts, measure the largest, and report the full DFS visiting order across the whole network.

### Concepts Covered

**Depth-First Search**

Recursive or stack-based flood fill through the graph

**Connected Components**

Maximal groups of mutually reachable vertices

**Largest Component**

Track the biggest district by vertex count

**Full Graph Coverage**

Iterate all vertices to cover every component

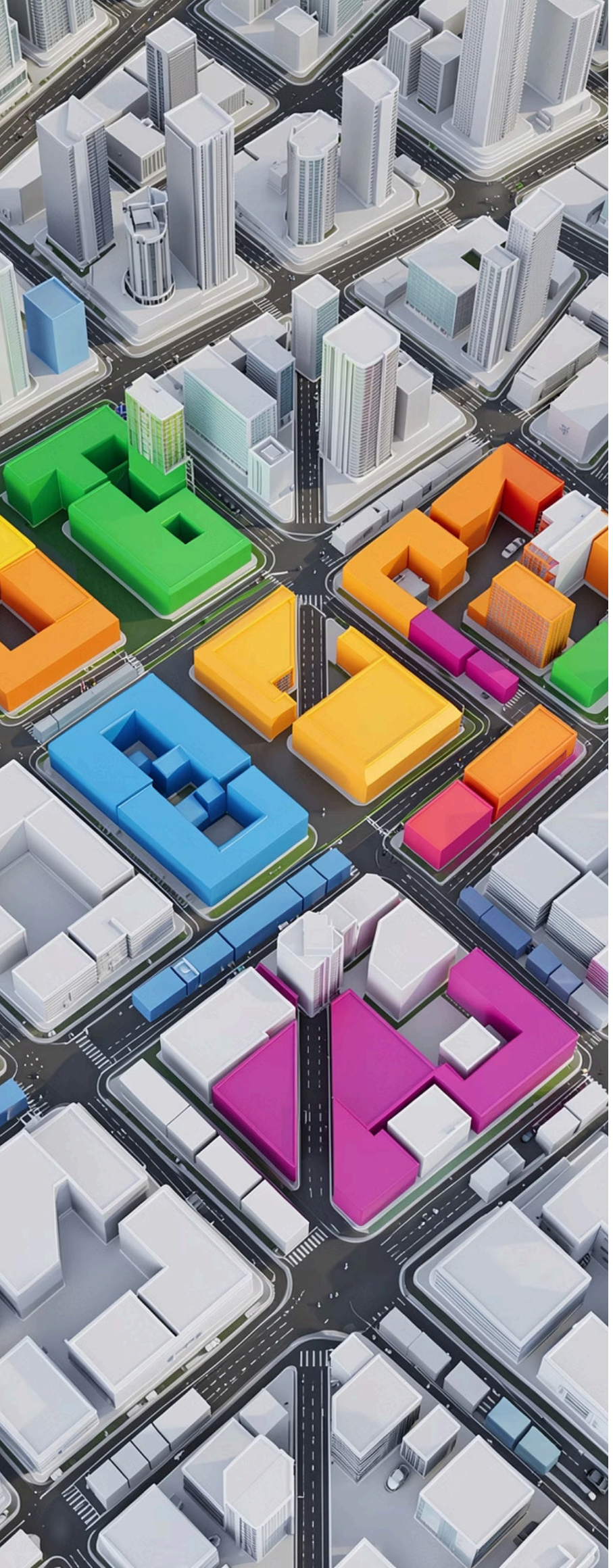
### Sample Run

Input:
7 4
0 1
0 2
2 3
4 5

Output:
Components: 3
Largest: 4
DFS Order: 0 1 2 3 4 5 6

⚠ Restart DFS from **every** unvisited vertex in order; running it only from vertex 0 misses every other district and undercounts components.

ⓘ Wrapping a DFS in a loop over all vertices turns "explore from here" into "partition the whole graph" — each restart marks the boundary of a new connected component.



LEVEL 4

# City Blocks

## Matrix as Graph (Grid Components) · ★★★★ Advanced

Switch to the districts map: a grid where **1 is a city block** and **0 is open ground**. Treat each block as a graph node connected to its four neighbours, then count the districts (connected groups of blocks), measure the largest, and total the occupied blocks.

### Grid as Implicit Graph

Each cell is a node; up/down/left/right neighbours are edges

### 4-Directional Adjacency

Connect only orthogonal neighbours — no diagonals

### Flood Fill BFS

Scan in row-major order; flood-fill each unseen block cell

### Count & Size

Track number of districts, largest district, and total block cells

# Level 4 — Problem Details

## Problem Statement

**Given:** A line `R C`, then R rows of C integers (1 = block, 0 = open)

**Required output:**

- The number of districts (4-connected groups of 1s)
- The size of the largest district
- The total number of block cells

## Formula Guidance

For each unseen block cell: flood-fill its 4-connected group, counting its size
`districts` = number of flood fills
`cells` = total number of 1s

⚠ Connect only the four orthogonal neighbours and bounds-check every move; sliding off the edge or counting diagonals merges districts that should stay separate.

## Sample Run

**Input:**

```
3 4
1 0 1 1
1 0 0 1
0 0 1 1
```

**Output:**

**Districts: 2**
**Largest: 5**
**Cells: 7**

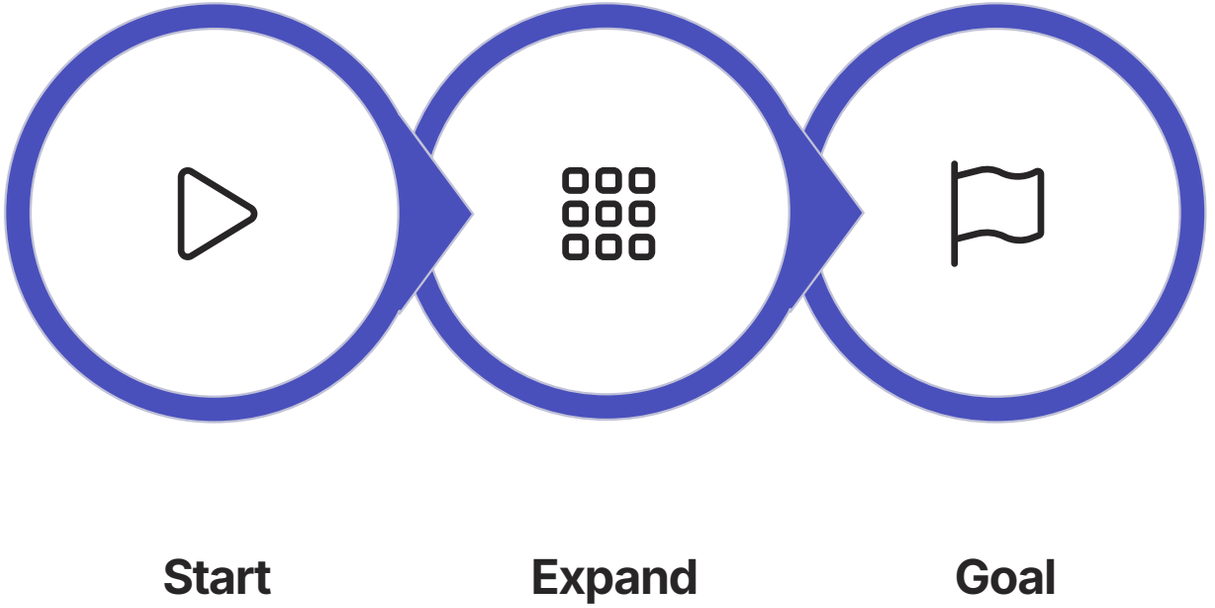
ⓘ A grid is just a graph in disguise: each cell is a node and its up/down/left/right neighbours are edges, so the same flood-fill that finds graph components counts grid regions.

LEVEL 5

# Cross Town

## Grid Shortest Path (BFS on a Matrix) · ★★★★ Advanced

Route a vehicle across the open streets. On the same kind of grid (0 is open, 1 is blocked) find the shortest number of moves from the top-left entrance to the bottom-right exit, moving only through open cells in the four directions.




BFS on a grid is the unweighted shortest-path algorithm applied to an implicit graph — every open cell is a node and each legal move is a unit-cost edge.

# Level 5 — Problem Details

## Problem Statement

**Given:** A line `R C`, then R rows of C integers (0 = open, 1 = blocked)

**Required output:**

- The shortest number of moves from (0,0) to (R-1,C-1), or -1 if blocked
- Whether the exit is reachable (Yes or No)

## Constraints

- Move only through open (0) cells, four directions
- The start cell is distance 0
- If the start or exit is blocked, or no path exists, the distance is -1

## Formula Guidance

BFS from (0,0) over open cells; `dist[(0,0)] = 0`
Each neighbour move costs 1 step
Answer = `dist[(R-1, C-1)]` (or -1 if never set)

⚠ Keep the start cell at distance 0 — the answer is a count of moves (edges); seeding it at 1 reports a distance one too large everywhere.

## Sample Run

Input:
4 4
0 0 1 0
1 0 1 0
1 0 0 0
1 1 1 0

Output:
Distance: 6
Reachable: Yes

ⓘ BFS on a grid is the unweighted shortest-path algorithm applied to an implicit graph — every open cell is a node and each legal move is a unit-cost edge.

LEVEL 6 — EXPERT

# Network Audit

## Components, Cycles, Bipartite & Reach (Capstone) · ★★★★★ Expert

Close the engine with a full network audit. On the road graph, report the number of districts (components), whether the network contains a cycle, whether it is bipartite (two-colourable), and the farthest hop distance reachable from intersection 0.

1

### Connected Components

Count DFS/BFS restarts over all vertices to find all districts

2

### Cycle Detection

A visited non-parent neighbour in undirected DFS signals a cycle

3

### Bipartite Check

2-colour via BFS; a same-colour edge breaks bipartiteness

4

### BFS Eccentricity

Maximum BFS distance from vertex 0 among all reachable vertices

# Level 6 — Problem Details

## Problem Statement

**Given:** A graph `N M` with M undirected edges

**Required output:**

- The number of connected components
- Whether the graph has a cycle (Yes or No)
- Whether the graph is bipartite (Yes or No)
- The farthest hop distance from vertex 0

## Formula Guidance

`components:` count DFS/BFS restarts over all vertices
`undirected cycle:` a visited non-parent neighbour signals a cycle
`bipartite:` 2-colour via BFS; same-colour edge breaks it
`farthest:` max BFS distance from vertex 0

⚠ In undirected cycle detection, skip the single edge back to the parent; treating that edge as a back-edge reports a cycle in every tree.

## Sample Run

Input:
6 3
0 1
1 2
3 4

Output:
Components: 3
Has Cycle: No
Bipartite: Yes
Farthest: 2

ⓘ A network audit composes the core traversals: the same DFS/BFS skeleton, augmented with a parent check, a colour array, or a distance array, answers four different structural questions.

# Metropolis — Key Thinking Skills

Each level builds on the last, composing fundamental graph algorithms into a complete city routing engine. Here are the core insights to carry forward:

### Adjacency List is the Foundation

Once built correctly (both endpoints for undirected edges), structural facts like degree, max degree, and isolation are simple reads over the lists.

### BFS = Free Shortest Paths

Because BFS expands in distance order, the first visit to any vertex is always along a minimum-hop route in an unweighted graph.

### DFS Loop = Full Partition

Wrapping DFS in a loop over all vertices turns "explore from here" into "partition the whole graph" — each restart marks a new connected component.

### Grid = Graph in Disguise

Every cell is a node; orthogonal neighbours are edges. The same BFS/DFS algorithms apply directly to grids for flood fill and shortest path.

### Compose Traversals for Audits

A single DFS/BFS skeleton, augmented with a parent check, colour array, or distance array, answers multiple structural questions simultaneously.